Video Classification with 2D and 3D Convolutional Networks

Kolbeinn Tumi Sindrason
s253026@dtu.dk

1. Introduction

Video classification assigns an action label to a sequence of frames. Individual frames provide appearance and pose cues, while changes across frames can reveal motion. This project studies how the way a network combines frames affects recognition on a ten-class workout subset of UCF101 [2] supplied by the course [3].

We compare three 2D CNN approaches and one 3D CNN, using compact models trained from scratch. The project also examines subject leakage through a comparison using corrected splits and an RGB/optical-flow dual-stream model. We report the completed single-stream cluster runs on the original splits. The corrected-split and dual-stream comparisons remain to be completed.

2. Data and evaluation protocol

The original metadata contain 500 training, 120 validation, and 120 test videos across ten workout classes. Each video is represented by ten uniformly sampled RGB frames supplied with the dataset. We preserve their temporal order, resize each frame to 224×224 pixels, and scale pixel values to $[0, 1]$. The baseline preprocessing uses no data augmentation or channel normalization, and the models use no externally pretrained weights. Class indices are read from the supplied metadata.

Accuracy is evaluated using one prediction per video. Checkpoints are selected by highest validation video accuracy, with lower validation loss breaking ties. The test set is reserved for final evaluation and has not been used to select these models; test results are not yet available. We use the four cluster runs with a common 50-epoch maximum, excluding smoke tests, laptop pilots, and earlier shorter cluster runs.

The assignment identifies subject overlap in the original splits: different clips of the same person performing an action can occur across training and evaluation sets [3]. Checking for duplicate video identifiers alone cannot establish subject independence. Original-split accuracy therefore does not establish performance on unseen people; the corrected dataset is needed for that comparison.

3. Method

Shared 2D backbone. The per-frame encoder contains three 3×3 convolutions with 32, 64, and 128 output channels. Each convolution uses stride one and padding one, followed by ReLU and 2×2 max pooling. Global spatial average pooling produces a 128-dimensional feature vector, and a linear layer maps it to ten class logits. The baseline models use no normalization layers. Let $h(x_t)$ denote the feature vector of frame t, and let $z_t = Wh(x_t) + b$ denote its logits.

Per-frame aggregation. Each frame is trained with its video's action label using cross-entropy. At inference, the video prediction averages the ten frame probability vectors:

$$p_{\text{frame}}(y \mid V) = \frac{1}{T} \sum_{t=1}^{T} \text{softmax}(z_t)_y, \qquad T = 10. \quad (1)$$

The predicted class is the maximum-probability class. Validation loss is the negative log-likelihood of this averaged distribution. The model ignores frame order.

Late fusion. The same encoder processes all frames with shared weights. Their features are averaged before the linear classifier:

$$p_{\text{late}}(y \mid V) = \text{softmax}\left(W \frac{1}{T} \sum_{t=1}^{T} h(x_t) + b \right)_y . \quad (2)$$

Training uses cross-entropy on one logit vector per video. With this linear classifier, averaging features is equivalent to averaging logits, but is generally different from averaging probabilities. Mean fusion also ignores frame order. Its parameter count equals that of the per-frame model.

Early fusion. The ten RGB frames are stacked into a 30-channel image in temporal order. The first convolution is widened to accept these channels; the rest of the 2D backbone is unchanged. This allows frames to interact from the first layer and makes their ordering relevant. Time is represented by channels, so the network requires a fixed number of frames and does not slide a convolutional kernel along time.

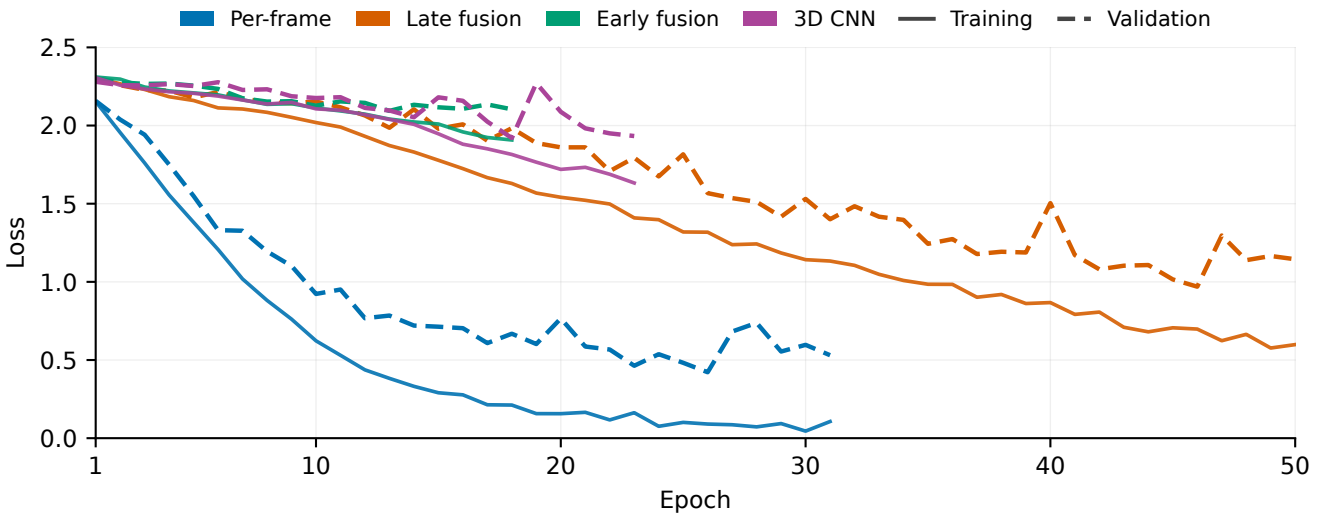


Figure 1. Training (solid) and validation (dashed) loss for all four cluster runs on one set of axes (224×224, seed 42, original splits). Colour identifies the model. Curves show unsmoothed epoch means and end when training stops. Per-frame training uses frame cross-entropy (CE); validation uses video negative log-likelihood after probability averaging. The other models use video CE for both splits. Epochs represent different numbers of optimizer updates.

3D CNN. The clip retains its $3 \times T \times H \times W$ representation. Three $3 \times 3 \times 3$ convolutions with padding one produce 32, 64, and 128 channels. Each is followed by ReLU and $1 \times 2 \times 2$ max pooling, preserving temporal resolution while reducing spatial resolution. Global spatiotemporal average pooling and a linear classifier produce video logits. The temporal kernels learn local interactions between the sampled frames. Both early fusion and the 3D CNN use video-level cross-entropy.

4. Experiments and discussion

4.1. Single-stream comparison (Task 4.1)

All four models are trained from random initialization using Adam with learning rate 10^{-3}, seed 42, and a maximum of 50 epochs. The cluster runs use PyTorch 2.8.0 with CUDA on an NVIDIA A100 GPU partition (MIG 1g.10gb). Early stopping monitors validation video loss with patience five and a minimum improvement of 10^{-4}. Stopping and checkpoint selection are distinct: we report the highest-accuracy checkpoint, with lower validation loss breaking ties, rather than automatically taking the last epoch or the lowest-loss checkpoint.

Per-frame training uses batches of eight frames, giving 625 optimizer updates per epoch from 5,000 frames. Late and early fusion use batches of two videos (250 updates per epoch), while the 3D CNN uses one video per batch (500 updates). Validation uses one video per batch for every model. Per-frame, early fusion, and the 3D CNN stop after 31, 18, and 23 epochs; late fusion reaches the 50-epoch limit. Total updates are therefore 19,375, 12,500, 4,500, and 11,500 for per-frame, late fusion, early fusion, and 3D CNN, respectively. This is a comparison under a shared stopping rule, not equal optimization or computation budgets.

Table 1. Original-split validation results from the cluster runs. Epochs denotes selected/completed epochs. Accuracy and loss come from the same accuracy-selected checkpoint, not the final epoch. No test results are included.

Model	Epochs	Val. (%)	Val. loss
Per-frame	22/31	90.8	0.567
Late fusion	46/50	75.0	0.969
Early fusion	18/18	22.5	2.104
3D CNN	22/23	32.5	1.950

Per-frame aggregation achieves the highest validation accuracy at 90.8% (109/120 videos), followed by late fusion at 75.0% (90/120), the 3D CNN at 32.5% (39/120), and early fusion at 22.5% (27/120). Table 1 reports accuracy and loss at the same selected checkpoint. For example, per-frame accuracy peaks at epoch 22, whereas its lowest validation loss is 0.422 at epoch 26, with 88.3% accuracy. Thus, minimum loss and maximum accuracy need not identify the same epoch.

Learning curves. Figure 1 places all training and validation losses on one set of axes. Per-frame loss falls fastest, but validation loss fluctuates above its epoch-26 minimum before stopping. Late fusion improves over a longer pe-

riod: its validation loss falls from 2.295 to 0.969 at epoch 46, then rises to 1.145 by epoch 50 while training loss continues to decrease. This late divergence is consistent with some overfitting. Early fusion and the 3D CNN finish with training losses of 1.907 and 1.633 and training accuracies of 29.0% and 41.6%, respectively. Their weak validation performance coincides with incomplete training-set fit; these runs do not establish that the architectures lack useful temporal information. In particular, loss-based early stopping can halt training despite continued improvement in accuracy, as in early fusion.

Per-frame training loss is frame-level cross-entropy, whereas validation loss is video-level negative log-likelihood after probability averaging. Their gap is not a like-for-like generalization gap. The other models use video cross-entropy for both splits. Training losses average batches during weight updates; validation uses the end-of-epoch model. All results use one seed and the original splits with subject overlap. They identify per-frame aggregation as the current candidate for the corrected-split experiment, but cannot establish its performance on unseen subjects. Test results remain pending.

4.2. Corrected-split comparison (Task 4.2.1)

Draft status: corrected-split results are pending. Per-frame aggregation is selected using the original-split validation results. Retrain it from random initialization on the supplied corrected training split, keeping the architecture, preprocessing, learning rate, seed, and training budget matched to the original-split experiment. Use each regime's validation set for checkpoint selection, then evaluate the selected checkpoints on their respective test sets. Record the corrected split sizes and report accuracy changes in percentage points. If the training recipe changes, rerun the original baseline under that recipe too.

A drop would be consistent with optimistic evaluation under subject overlap, but its magnitude must be measured. The split populations also change, so the comparison is not a paired experiment on identical test videos. No corrected-split score or leakage-related drop is claimed here.

4.3. RGB and optical flow (Task 4.2.2)

Draft status: dual-stream implementation and results are pending. Two-stream networks combine an appearance stream with a motion stream based on optical flow [1]. The corrected course dataset supplies flow generated using RAFT [3, 4]. Before finalizing the flow encoder, inspect the supplied representation, its channel count, and its alignment with the RGB frames. In particular, colored flow PNGs must not be described as raw horizontal and vertical displacement fields without verifying their encoding.

The remaining experiment will compare RGB-only, flow-only, and fused predictions on the corrected splits. Document the actual encoders, normalization, fusion operation, and training schedule, and select any fusion weight using validation data. Train any source encoder on corrected training data as well. Report final test results only after model selection; the current draft makes no claim that optical flow improves accuracy.

5. Conclusion

Per-frame aggregation achieves the highest original-split validation accuracy in the cluster runs (90.8%), followed by late fusion (75.0%). Early fusion and the 3D CNN fit the training data less well under the chosen optimization and stopping settings. These single-seed results do not isolate architectural effects from training-budget differences, and subject leakage limits their interpretation. Held-out test evaluation, the corrected-split comparison, and the contribution of optical flow remain necessary before drawing conclusions about generalization.

References

[1] Karen Simonyan and Andrew Zisserman. Two-stream convolutional networks for action recognition in videos. In *Advances in Neural Information Processing Systems*, 2014. 3

[2] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. UCF101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402*, 2012. 1

[3] Technical University of Denmark. Video classification: Project 2. Introduction to Deep Learning in Computer Vision, course assignment, 2026. 1, 3

[4] Zachary Teed and Jia Deng. RAFT: Recurrent all-pairs field transforms for optical flow. In *European Conference on Computer Vision*, 2020. 3

Statement on use of generative AI

Draft disclosure; the author must review and complete this statement before submission.

OpenAI Codex assisted with project scaffolding, configuration files, data-loading and model implementations, the training and checkpoint-selection code, automated checks, experiment analysis, and report preparation. Report assistance included drafting and editing prose, producing a comparison table and a learning-curve figure from saved logs, locating references, and checking the compiled document against the page limit.

For this draft, numerical results were checked against saved run summaries, histories, and effective configurations; model descriptions were checked against the implementation. Missing experiment results are explicitly identified. Before submission, the author should verify the final code, results, references, and interpretations, and update this statement to include any other tools or uses and the checks they personally performed.